

LLD Problems — Coded Solutions

In This Edition

1	The Machine-Coding Approach	2
	1.1 Time-box strategy (45--90 min round)	2
	1.2 What interviewers grade (roughly in priority order)	2
	1.3 What to deprioritize under time pressure	2
	1.4 The noun-verb extraction recipe	2
2	1. LRU Cache (+ LFU Variant Note)	2
	2.1 Requirements	2
	2.2 Core entities / classes	3
	2.3 Class diagram (ASCII UML)	3
	2.4 Design patterns used	3
	2.5 Full code	3
	2.6 Extensibility discussion	5
3	2. Parking Lot	6
	3.1 Requirements	6
	3.2 Core entities / classes	6
	3.3 Class diagram (ASCII UML)	6
	3.4 Design patterns used	7
	3.5 Full code	7
	3.6 Extensibility discussion	12
4	3. Rate Limiter	12
	4.1 Requirements	12
	4.2 Core entities / classes	13
	4.3 Class diagram (ASCII UML)	13
	4.4 Design patterns used	13
	4.5 Full code	13
	4.6 Extensibility discussion	17
5	4. Splitwise / Expense Sharing	17
	5.1 Requirements	17
	5.2 Core entities / classes	17
	5.3 Class diagram (ASCII UML)	18
	5.4 Design patterns used	18
	5.5 Full code	18
	5.6 Extensibility discussion	22
6	5. Elevator System	22
	6.1 Requirements	22
	6.2 Core entities / classes	23
	6.3 Class diagram (ASCII UML)	23
	6.4 Design patterns used	23
	6.5 Full code	24
	6.6 Extensibility discussion	28
7	6. Tic-Tac-Toe (Clean OO Board Game)	29
	7.1 Requirements	29
	7.2 Core entities / classes	29
	7.3 Class diagram (ASCII UML)	29
	7.4 Design patterns used	30
	7.5 Full code	30
	7.6 Extensibility discussion	34
8	Common Patterns Across LLD Problems	34
	8.1 Entity-extraction checklist	34
	8.2 Which pattern fits which need	34
	8.3 Revision cheat sheet	35
	8.4 The five questions interviewers ask at the end	35

How to use this file: Each ## section is a self-contained LLD problem. Read Requirements + Class Diagram first, then study the code. Run any block directly with `python3`. The closing sections distill the cross-cutting patterns for rapid revision.

1 The Machine-Coding Approach

1.1 Time-box strategy (45--90 min round)

Phase	Time	Goal
Clarify requirements	5 min	Pin scope; avoid building the wrong thing
Extract entities & relationships	5 min	Noun-verb extraction on whiteboard
Sketch class diagram	5–10 min	Get agreement before coding
Code core classes + happy path	20–30 min	Must compile/run
Add edge cases + extensibility	10 min	Show you think beyond happy path
Demo driver	5 min	Prove it works end-to-end

1.2 What interviewers grade (roughly in priority order)

1. **Working, runnable code** — broken code is an automatic downgrade.
2. **Clean abstractions** — classes with single responsibilities; meaningful names.
3. **Extensibility** — adding a new feature should require adding code, not changing existing code (OCP).
4. **Edge cases handled** — not just happy path (full parking lot, divide-by-zero in splits, empty elevator queue).
5. **Design patterns named** — shows vocabulary; don't force patterns but do name what you use.

1.3 What to deprioritize under time pressure

- › Persistence / DB layer (mention it, skip it).
- › Concurrency / thread-safety (mention locks, skip implementation unless asked).
- › Perfect error messages (simple exceptions are fine).
- › Over-engineering: 3-layer factory + abstract-factory combos for a 45-min round are a red flag.

1.4 The noun-verb extraction recipe

From the problem statement: circle **nouns** → candidate classes; underline **verbs** → candidate methods. Then prune: nouns that are just attributes (e.g. "name", "price") become fields, not classes.

2 1. LRU Cache (+ LFU Variant Note)

2.1 Requirements

Functional

- › `get(key)` — return value or -1 if not present.
- › `put(key, value)` — insert; if capacity exceeded, evict **least recently used** entry.

- › Both operations must be **O(1)**.

Clarifying questions to ask

- › Is capacity fixed at construction? (Yes.)
- › Thread-safe? (Assume single-threaded unless asked; mention `threading.Lock` for MT.)
- › Key/value types? (Generics / Any for flexibility.)
- › On put of existing key, update value and recency? (Yes.)

2.2 Core entities / classes

Class	Responsibility
DLinkedListNode	Doubly-linked list node (key, value, prev, next pointers)
LRUCache	HashMap + doubly-linked list; exposes get/put

2.3 Class diagram (ASCII UML)



2.4 Design patterns used

- › **No classic GoF pattern** — this is a pure data-structure problem. The key insight is the **sentinel head/tail** trick to avoid null checks. Interviewers want to see you know this.
- › Mention: for thread-safety you'd wrap with a `threading.Lock` OR use `collections.OrderedDict` (which Python's `stdlib` provides for a simpler implementation, but interviewers want the manual version to test understanding).

2.5 Full code

```

1 from __future__ import annotations
2 from typing import Optional
3
4
5 class DLinkedListNode:
6     """Doubly-linked list node used as the internal building block."""
7
8     def __init__(self, key: int = 0, value: int = 0) -> None:
9         self.key = key
10        self.value = value
11        self.prev: Optional[DLinkedListNode] = None
12        self.next: Optional[DLinkedListNode] = None
  
```

```

13
14
15 class LRUCache:
16     """
17     O(1) get/put LRU Cache implemented with a HashMap + doubly-linked list.
18
19     Invariant: the list is ordered most-recently-used (front) to
20     least-recently-used (back). Sentinel head/tail nodes eliminate
21     edge-case null checks.
22     """
23
24     def __init__(self, capacity: int) → None:
25         if capacity ≤ 0:
26             raise ValueError("Capacity must be positive")
27         self.capacity = capacity
28         self.cache: dict[int, DLinkedNode] = {}
29
30         # Sentinels — never evicted, never returned
31         self.head = DLinkedNode() # MRU sentinel
32         self.tail = DLinkedNode() # LRU sentinel
33         self.head.next = self.tail
34         self.tail.prev = self.head
35
36         # ----- #
37         # Public API                                     #
38         # ----- #
39
40     def get(self, key: int) → int:
41         if key not in self.cache:
42             return -1
43         node = self.cache[key]
44         self._move_to_front(node)
45         return node.value
46
47     def put(self, key: int, value: int) → None:
48         if key in self.cache:
49             node = self.cache[key]
50             node.value = value
51             self._move_to_front(node)
52         else:
53             node = DLinkedNode(key, value)
54             self.cache[key] = node
55             self._add_to_front(node)
56             if len(self.cache) > self.capacity:
57                 lru = self._pop_from_back()
58                 del self.cache[lru.key]
59
60         # ----- #
61         # Private helpers                                     #
62         # ----- #
63
64     def _add_to_front(self, node: DLinkedNode) → None:
65         """Insert node right after head sentinel (= most-recently-used)."""
66         node.prev = self.head
67         node.next = self.head.next
68         self.head.next.prev = node
69         self.head.next = node
70
71     def _remove(self, node: DLinkedNode) → None:
72         """Unlink node from its current position."""
73         node.prev.next = node.next

```

```

74     node.next.prev = node.prev
75
76     def _move_to_front(self, node: DLinkedNode) → None:
77         self._remove(node)
78         self._add_to_front(node)
79
80     def _pop_from_back(self) → DLinkedNode:
81         """Remove and return the LRU node (just before tail sentinel)."""
82         lru = self.tail.prev
83         self._remove(lru)
84         return lru
85
86
87 # ----- #
88 # LFU variant – brief notes (full implementation would add freq map) #
89 # ----- #
90 # LFU Cache (Least Frequently Used):
91 #   - Evict the entry with the LOWEST access frequency;
92 #     tie-break on LRU order within same frequency.
93 # Data structures needed:
94 #   - key_to_val: dict[key → value]
95 #   - key_to_freq: dict[key → frequency]
96 #   - freq_to_keys: dict[freq → OrderedDict[key → None]] (ordered = LRU within freq)
97 #   - min_freq: int (track current minimum)
98 # On get/put: increment freq, move key to freq+1 bucket, update min_freq.
99 # All operations O(1). See Python's collections.OrderedDict for ordered bucket.
100
101
102 if __name__ == "__main__":
103     cache = LRUCache(capacity=3)
104     cache.put(1, 10)
105     cache.put(2, 20)
106     cache.put(3, 30)
107
108     print(cache.get(1)) # 10 - 1 is now MRU
109     cache.put(4, 40) # evicts 2 (LRU)
110     print(cache.get(2)) # -1 - evicted
111     print(cache.get(3)) # 30
112     cache.put(5, 50) # evicts 4 (LRU after 1 and 3 were accessed)
113     print(cache.get(4)) # -1
114     print(cache.get(1)) # 10
115     print(cache.get(3)) # 30
116     print(cache.get(5)) # 50

```

2.6 Extensibility discussion

- **TTL / expiry:** Add expiry: Optional[float] to DLinkedNode. On get, check time.time() > expiry and treat as miss + remove. A background thread or lazy expiry on access both work.
- **Thread-safety:** Wrap get/put with threading.Lock. For higher concurrency, shard the cache into N sub-caches keyed by hash(key) % N.
- **Generics:** Replace int with TypeVar('K') / TypeVar('V') and reuse across types.

Interview takeaway: The sentinel head/tail trick is the key insight. Without sentinels you need if node.prev null-checks everywhere — messy and error-prone. Name it explicitly.

3 2. Parking Lot

3.1 Requirements

Functional

- › Support multiple levels, multiple spot types (COMPACT, LARGE, MOTORCYCLE, ELECTRIC).
- › Vehicles: Motorcycle, Car, Truck — each fits specific spot types.
- › `park(vehicle)` — find nearest available spot; return ticket.
- › `unpark(ticket)` — free the spot; compute fee.
- › Pluggable pricing strategy (hourly, flat-rate, etc.).
- › Display available spots per level.

Clarifying questions to ask

- › Can a car park in a large spot if compact is full? (Yes, with preference ordering.)
- › Is pricing per hour or per entry? (Make it a strategy — both.)
- › Multiple entrances? (Assume single for now; mention extension.)
- › Reservations / pre-booking? (Out of scope.)
- › Thread-safety for concurrent `park` calls? (Mention lock per level; skip for now.)

3.2 Core entities / classes

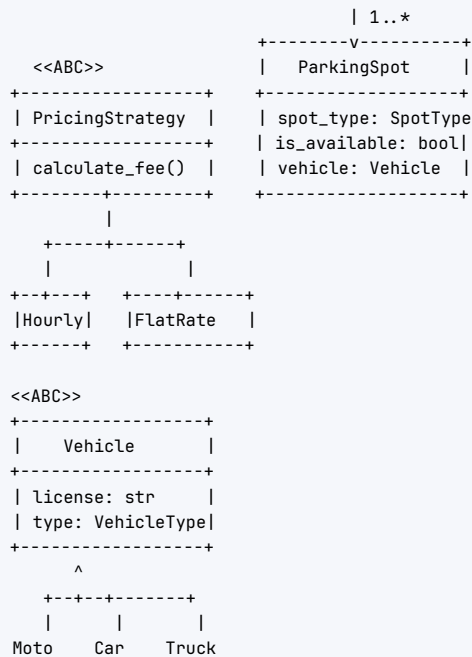
Class	Responsibility
VehicleType (enum)	MOTORCYCLE, CAR, TRUCK
SpotType (enum)	MOTORCYCLE, COMPACT, LARGE, ELECTRIC
Vehicle (ABC)	Base vehicle with type + license plate
ParkingSpot	Individual spot — type, level, number, occupied flag
ParkingLevel	Collection of spots on one floor; find-spot logic
ParkingLot	Singleton; owns levels; exposes park/unpark
Ticket	Issued on park; holds spot ref + entry time
PricingStrategy (ABC)	Strategy interface: <code>calculate_fee(ticket) → float</code>
HourlyPricing	Concrete strategy
FlatRatePricing	Concrete strategy

3.3 Class diagram (ASCII UML)

```

+-----+
|  ParkingLot  |  <<Singleton>>
+-----+
| levels: List[Level] |
| strategy: PricingStrategy |
+-----+
| park(vehicle)      |
| unpark(ticket)     |
+-----+
| 1..*
+-----v-----+
|  ParkingLevel  |
+-----+
| level_num: int   |
| spots: List[Spot] |
+-----+
| find_spot(vehicle)|
+-----+

```



3.4 Design patterns used

- **Strategy** — PricingStrategy lets you swap hourly/flat-rate/weekend pricing without touching ParkingLot.
- **Singleton** — ParkingLot represents a single physical lot; use `__new__` or a module-level instance.
- **Factory Method** — VehicleFactory.create(type, plate) to instantiate vehicles without if/else in client code.
- **Template Method** — Vehicle defines allowed_spot_types() as an abstract method; each subclass overrides it.

3.5 Full code

```

1  from __future__ import annotations
2
3  import time
4  from abc import ABC, abstractmethod
5  from enum import Enum, auto
6  from typing import Optional
7  from dataclasses import dataclass, field
8
9
10 # ----- #
11 # Enums                                     #
12 # ----- #
13
14 class VehicleType(Enum):
15     MOTORCYCLE = auto()
16     CAR = auto()
17     TRUCK = auto()
18
19
20 class SpotType(Enum):
21     MOTORCYCLE = auto()

```

```

22     COMPACT = auto()
23     LARGE = auto()
24     ELECTRIC = auto()
25
26
27 # ----- #
28 # Vehicles                                     #
29 # ----- #
30
31 class Vehicle(ABC):
32     def __init__(self, license_plate: str, vehicle_type: VehicleType) → None:
33         self.license_plate = license_plate
34         self.vehicle_type = vehicle_type
35
36     @abstractmethod
37     def allowed_spot_types(self) → list[SpotType]:
38         """Ordered preference: first = most preferred spot type."""
39         ...
40
41     def __repr__(self) → str:
42         return f"{self.__class__.__name__}({self.license_plate})"
43
44
45 class Motorcycle(Vehicle):
46     def __init__(self, license_plate: str) → None:
47         super().__init__(license_plate, VehicleType.MOTORCYCLE)
48
49     def allowed_spot_types(self) → list[SpotType]:
50         return [SpotType.MOTORCYCLE, SpotType.COMPACT, SpotType.LARGE]
51
52
53 class Car(Vehicle):
54     def __init__(self, license_plate: str) → None:
55         super().__init__(license_plate, VehicleType.CAR)
56
57     def allowed_spot_types(self) → list[SpotType]:
58         return [SpotType.COMPACT, SpotType.LARGE, SpotType.ELECTRIC]
59
60
61 class Truck(Vehicle):
62     def __init__(self, license_plate: str) → None:
63         super().__init__(license_plate, VehicleType.TRUCK)
64
65     def allowed_spot_types(self) → list[SpotType]:
66         return [SpotType.LARGE]
67
68
69 class VehicleFactory:
70     """Factory to decouple client from concrete vehicle classes."""
71
72     _registry: dict[VehicleType, type[Vehicle]] = {
73         VehicleType.MOTORCYCLE: Motorcycle,
74         VehicleType.CAR: Car,
75         VehicleType.TRUCK: Truck,
76     }
77
78     @classmethod
79     def create(cls, vehicle_type: VehicleType, license_plate: str) → Vehicle:
80         klass = cls._registry.get(vehicle_type)
81         if klass is None:
82             raise ValueError(f"Unknown vehicle type: {vehicle_type}")

```

```

83         return klass(license_plate)
84
85
86 # ----- #
87 # Parking Spot #
88 # ----- #
89
90 class ParkingSpot:
91     def __init__(self, spot_id: str, spot_type: SpotType, level: int) → None:
92         self.spot_id = spot_id
93         self.spot_type = spot_type
94         self.level = level
95         self._vehicle: Optional[Vehicle] = None
96
97     @property
98     def is_available(self) → bool:
99         return self._vehicle is None
100
101     def park(self, vehicle: Vehicle) → None:
102         if not self.is_available:
103             raise RuntimeError(f"Spot {self.spot_id} already occupied")
104         self._vehicle = vehicle
105
106     def unpark(self) → Vehicle:
107         if self.is_available:
108             raise RuntimeError(f"Spot {self.spot_id} is empty")
109         vehicle = self._vehicle
110         self._vehicle = None
111         return vehicle # type: ignore[return-value]
112
113     def __repr__(self) → str:
114         status = "FREE" if self.is_available else f"OCC({self._vehicle})"
115         return f"Spot[{self.spot_id},{self.spot_type.name},{status}]"
116
117 # ----- #
118 # Ticket #
119 # ----- #
120
121
122 @dataclass
123 class Ticket:
124     ticket_id: str
125     vehicle: Vehicle
126     spot: ParkingSpot
127     entry_time: float = field(default_factory=time.time)
128     exit_time: Optional[float] = None
129
130     def duration_hours(self) → float:
131         end = self.exit_time or time.time()
132         return (end - self.entry_time) / 3600.0
133
134 # ----- #
135 # Pricing Strategy #
136 # ----- #
137
138
139 class PricingStrategy(ABC):
140     @abstractmethod
141     def calculate_fee(self, ticket: Ticket) → float:
142         ...
143

```

```

144
145 class HourlyPricing(PricingStrategy):
146     def __init__(self, rate_per_hour: float = 2.0) → None:
147         self.rate_per_hour = rate_per_hour
148
149     def calculate_fee(self, ticket: Ticket) → float:
150         import math
151         hours = math.ceil(ticket.duration_hours()) # round up to next hour
152         return max(hours, 1) * self.rate_per_hour
153
154
155 class FlatRatePricing(PricingStrategy):
156     def __init__(self, flat_fee: float = 5.0) → None:
157         self.flat_fee = flat_fee
158
159     def calculate_fee(self, ticket: Ticket) → float:
160         return self.flat_fee
161
162
163 # ----- #
164 # Parking Level #
165 # ----- #
166
167 class ParkingLevel:
168     def __init__(self, level_num: int, spots: list[ParkingSpot]) → None:
169         self.level_num = level_num
170         self.spots = spots
171
172     def find_spot(self, vehicle: Vehicle) → Optional[ParkingSpot]:
173         """Return first available spot matching vehicle preference order."""
174         for preferred_type in vehicle.allowed_spot_types():
175             for spot in self.spots:
176                 if spot.spot_type == preferred_type and spot.is_available:
177                     return spot
178         return None
179
180     def available_count(self) → dict[SpotType, int]:
181         counts: dict[SpotType, int] = {}
182         for spot in self.spots:
183             if spot.is_available:
184                 counts[spot.spot_type] = counts.get(spot.spot_type, 0) + 1
185         return counts
186
187
188 # ----- #
189 # Parking Lot (Singleton) #
190 # ----- #
191
192 class ParkingLot:
193     _instance: Optional[ParkingLot] = None
194
195     def __new__(cls, *args, **kwargs) → ParkingLot:
196         if cls._instance is None:
197             cls._instance = super().__new__(cls)
198         return cls._instance
199
200     def __init__(
201         self,
202         name: str,
203         levels: list[ParkingLevel],
204         pricing_strategy: Optional[PricingStrategy] = None,

```

```

205     ) → None:
206         # Guard against re-init on subsequent __new__ returns
207         if hasattr(self, "_initialized"):
208             return
209         self.name = name
210         self.levels = levels
211         self.pricing_strategy: PricingStrategy = pricing_strategy or HourlyPricing()
212         self._tickets: dict[str, Ticket] = {}
213         self._ticket_counter = 0
214         self._initialized = True
215
216     def park(self, vehicle: Vehicle) → Ticket:
217         for level in self.levels:
218             spot = level.find_spot(vehicle)
219             if spot:
220                 spot.park(vehicle)
221                 self._ticket_counter += 1
222                 ticket = Ticket(
223                     ticket_id=f"T{self._ticket_counter:04d}",
224                     vehicle=vehicle,
225                     spot=spot,
226                 )
227                 self._tickets[ticket.ticket_id] = ticket
228                 print(f"   Parked {vehicle} at {spot.spot_id} (Level {level.level_num})")
229                 return ticket
230             raise RuntimeError("No available spot for this vehicle type")
231
232     def unpark(self, ticket_id: str) → float:
233         ticket = self._tickets.get(ticket_id)
234         if ticket is None:
235             raise ValueError(f"Unknown ticket: {ticket_id}")
236         ticket.exit_time = time.time()
237         ticket.spot.unpark()
238         fee = self.pricing_strategy.calculate_fee(ticket)
239         del self._tickets[ticket_id]
240         print(f"   Unparked {ticket.vehicle} | Fee: ${fee:.2f}")
241         return fee
242
243     def display_availability(self) → None:
244         print(f"\n--- {self.name} Availability ---")
245         for level in self.levels:
246             counts = level.available_count()
247             print(f"   Level {level.level_num}: {counts}")
248
249
250 # ----- #
251 # Helper: build a lot                               #
252 # ----- #
253
254 def build_parking_lot() → ParkingLot:
255     # Reset singleton for demo purposes
256     ParkingLot._instance = None
257
258     def make_spots(level: int, config: dict[SpotType, int]) → list[ParkingSpot]:
259         spots = []
260         for spot_type, count in config.items():
261             for i in range(1, count + 1):
262                 spot_id = f"L{level}-{spot_type.name[0]}{i}"
263                 spots.append(ParkingSpot(spot_id, spot_type, level))
264         return spots
265

```

```

266     levels = [
267         ParkingLevel(1, make_spots(1, {
268             SpotType.MOTORCYCLE: 5,
269             SpotType.COMPACT: 10,
270             SpotType.LARGE: 5,
271         })),
272         ParkingLevel(2, make_spots(2, {
273             SpotType.COMPACT: 8,
274             SpotType.LARGE: 4,
275             SpotType.ELECTRIC: 3,
276         })),
277     ]
278     return ParkingLot("City Center Garage", levels, HourlyPricing(rate_per_hour=3.0))
279
280
281 if __name__ == "__main__":
282     lot = build_parking_lot()
283     lot.display_availability()
284
285     moto = Motorcycle("MOTO-001")
286     car1 = Car("CAR-001")
287     truck = Truck("TRK-001")
288
289     t1 = lot.park(moto)
290     t2 = lot.park(car1)
291     t3 = lot.park(truck)
292     lot.display_availability()
293
294     # Simulate time passing (normally you'd actually wait)
295     t2.entry_time -= 7200 # pretend car parked 2 hours ago
296     lot.unpark(t2.ticket_id)
297     lot.unpark(t1.ticket_id)
298     lot.display_availability()

```

3.6 Extensibility discussion

- › **New vehicle type (e.g., Bus):** Add `VehicleType.BUS`, create class `Bus(Vehicle)` with `allowed_spot_types`, register in `VehicleFactory._registry`. Zero changes to `ParkingLot` or `ParkingLevel`.
- › **New pricing (weekend/dynamic):** Implement `WeekendPricing(PricingStrategy)`. Pass it at construction. OCP satisfied.
- › **EV charging:** Add `SpotType.ELECTRIC`; `Car.allowed_spot_types` already supports it. Add `ChargeSession` attached to `Ticket`.
- › **Reservations:** Add `reserve(vehicle, start_time)` that marks a spot with a future timestamp; `find_spot` skips reserved spots for the overlapping window.

Interview takeaway: Always name the Strategy pattern for pricing — it's the most common follow-up ("what if pricing changes on weekends?"). The answer is: new class, zero changes to existing code.

4 3. Rate Limiter

4.1 Requirements

Functional

- › `allow(client_id) → bool` — return True if request should be permitted.

- › Support Token Bucket, Sliding Window Log, Fixed Window Counter strategies.
- › Pluggable — caller picks strategy at construction.
- › Per-client limiting (different clients independent).

Clarifying questions to ask

- › Global limit or per-client? (Per-client.)
- › Burst tolerance? (Token Bucket allows bursts; fixed-window does not.)
- › Distributed (Redis-backed)? (Assume in-process for now; mention Redis extension.)
- › What happens on limit hit — queue or reject? (Reject / return False.)
- › Thread-safe? (Yes, per-client locks.)

4.2 Core entities / classes

Class	Responsibility
RateLimitStrategy (ABC)	Interface: <code>is_allowed(client_id) → bool</code>
TokenBucketStrategy	Refill tokens at fixed rate; allow if tokens > 0
FixedWindowStrategy	Count requests per fixed time window; reset on window roll
SlidingWindowLogStrategy	Keep timestamped log; count entries in last N seconds
RateLimiter	Context object; delegates to strategy

4.3 Class diagram (ASCII UML)



4.4 Design patterns used

- › **Strategy** — core pattern; swap algorithms at runtime.
- › **Factory / constructor injection** — `RateLimiter(strategy=TokenBucketStrategy(...))`.

4.5 Full code

```

1  from __future__ import annotations
2
3  import time
4  import threading
5  from abc import ABC, abstractmethod
  
```

```

6  from collections import deque
7  from dataclasses import dataclass, field
8  from typing import Optional
9
10
11 # ----- #
12 # Strategy Interface #
13 # ----- #
14
15 class RateLimitStrategy(ABC):
16     @abstractmethod
17     def is_allowed(self, client_id: str) → bool:
18         ...
19
20
21 # ----- #
22 # Strategy 1: Token Bucket #
23 # ----- #
24
25 @dataclass
26 class _TokenBucketState:
27     tokens: float
28     last_refill: float = field(default_factory=time.time)
29
30
31 class TokenBucketStrategy(RateLimitStrategy):
32     """
33     Each client starts with `capacity` tokens.
34     Tokens refill at `refill_rate` tokens/second.
35     Each request consumes 1 token. Allows bursts up to `capacity`.
36     """
37
38     def __init__(self, capacity: float, refill_rate: float) → None:
39         self.capacity = capacity
40         self.refill_rate = refill_rate
41         self._buckets: dict[str, _TokenBucketState] = {}
42         self._lock = threading.Lock()
43
44     def _get_bucket(self, client_id: str) → _TokenBucketState:
45         if client_id not in self._buckets:
46             self._buckets[client_id] = _TokenBucketState(tokens=self.capacity)
47         return self._buckets[client_id]
48
49     def is_allowed(self, client_id: str) → bool:
50         with self._lock:
51             now = time.time()
52             bucket = self._get_bucket(client_id)
53             elapsed = now - bucket.last_refill
54             bucket.tokens = min(
55                 self.capacity,
56                 bucket.tokens + elapsed * self.refill_rate
57             )
58             bucket.last_refill = now
59             if bucket.tokens ≥ 1:
60                 bucket.tokens -= 1
61             return True
62         return False
63
64
65 # ----- #
66 # Strategy 2: Fixed Window Counter #

```

```

67 # ----- #
68
69 @dataclass
70 class _FixedWindowState:
71     count: int = 0
72     window_start: float = field(default_factory=time.time)
73
74
75 class FixedWindowStrategy(RateLimitStrategy):
76     """
77     Allow up to `limit` requests per `window_seconds` window.
78     Window resets at the start of each period.
79     Weakness: burst at window boundary (2x limit in 2*epsilon seconds).
80     """
81
82     def __init__(self, limit: int, window_seconds: float) → None:
83         self.limit = limit
84         self.window_seconds = window_seconds
85         self._states: dict[str, _FixedWindowState] = {}
86         self._lock = threading.Lock()
87
88     def is_allowed(self, client_id: str) → bool:
89         with self._lock:
90             now = time.time()
91             if client_id not in self._states:
92                 self._states[client_id] = _FixedWindowState(window_start=now)
93                 state = self._states[client_id]
94
95             if now - state.window_start ≥ self.window_seconds:
96                 state.count = 0
97                 state.window_start = now
98
99             if state.count < self.limit:
100                 state.count += 1
101                 return True
102             return False
103
104 # ----- #
105 # Strategy 3: Sliding Window Log                                     #
106 # ----- #
107
108 class SlidingWindowLogStrategy(RateLimitStrategy):
109     """
110     Keep a deque of request timestamps per client.
111     On each request, prune entries older than `window_seconds`,
112     then check if remaining count < limit.
113     Most accurate; memory = O(limit) per client.
114     """
115
116     def __init__(self, limit: int, window_seconds: float) → None:
117         self.limit = limit
118         self.window_seconds = window_seconds
119         self._logs: dict[str, deque[float]] = {}
120         self._lock = threading.Lock()
121
122     def is_allowed(self, client_id: str) → bool:
123         with self._lock:
124             now = time.time()
125             if client_id not in self._logs:
126                 self._logs[client_id] = deque()

```

```

128         log = self._logs[client_id]
129
130         cutoff = now - self.window_seconds
131         while log and log[0] < cutoff:
132             log.popleft()
133
134         if len(log) < self.limit:
135             log.append(now)
136             return True
137         return False
138
139
140 # ----- #
141 # Rate Limiter Context #
142 # ----- #
143
144 class RateLimiter:
145     """
146     Context class. Delegates to the injected strategy.
147     Swap strategy at runtime with `set_strategy`.
148     """
149
150     def __init__(self, strategy: RateLimitStrategy) → None:
151         self._strategy = strategy
152
153     def set_strategy(self, strategy: RateLimitStrategy) → None:
154         self._strategy = strategy
155
156     def allow(self, client_id: str) → bool:
157         return self._strategy.is_allowed(client_id)
158
159
160 # ----- #
161 # Demo #
162 # ----- #
163
164 if __name__ == "__main__":
165     print("=== Token Bucket (capacity=5, refill=1/s) ===")
166     rl = RateLimiter(TokenBucketStrategy(capacity=5, refill_rate=1.0))
167     for i in range(7):
168         result = rl.allow("user_A")
169         print(f" Request {i+1}: {'ALLOWED' if result else 'DENIED'}")
170
171     print("\n=== Fixed Window (limit=3, window=10s) ===")
172     rl2 = RateLimiter(FixedWindowStrategy(limit=3, window_seconds=10))
173     for i in range(5):
174         result = rl2.allow("user_B")
175         print(f" Request {i+1}: {'ALLOWED' if result else 'DENIED'}")
176
177     print("\n=== Sliding Window Log (limit=3, window=1s) ===")
178     rl3 = RateLimiter(SlidingWindowLogStrategy(limit=3, window_seconds=1.0))
179     for i in range(4):
180         result = rl3.allow("user_C")
181         print(f" Request {i+1}: {'ALLOWED' if result else 'DENIED'}")
182     time.sleep(1.1)
183     result = rl3.allow("user_C")
184     print(f" After sleep: {'ALLOWED' if result else 'DENIED'}")

```

4.6 Extensibility discussion

- › **Redis-backed distributed limiter:** Replace the in-memory `_buckets/_logs` dicts with Redis calls using atomic `INCR/EXPIRE` or Lua scripts. Strategy interface unchanged.
- › **New algorithm (leaky bucket):** Implement `LeakyBucketStrategy(RateLimitStrategy)`. Inject it. No other changes.
- › **Middleware integration:** Wrap `RateLimiter.allow` in a decorator or WSGI/ASGI middleware. The core class is unaware of HTTP.
- › **Per-endpoint limits:** Key on `f"{client_id}:{endpoint}"` instead of just `client_id`.

Interview takeaway: Know the trade-offs: Token Bucket allows bursts; Fixed Window has boundary-burst weakness; Sliding Window Log is most accurate but costs $O(\text{limit})$ memory per client. Sliding Window Counter (hybrid) is the production sweet spot.

5 4. Splitwise / Expense Sharing

5.1 Requirements

Functional

- › Add users to groups.
- › Add an expense: payer, amount, split type (EQUAL, EXACT, PERCENT), participants.
- › `show_balances(user)` — who owes whom and how much.
- › `simplify_debts(group)` — minimize number of transactions to settle.

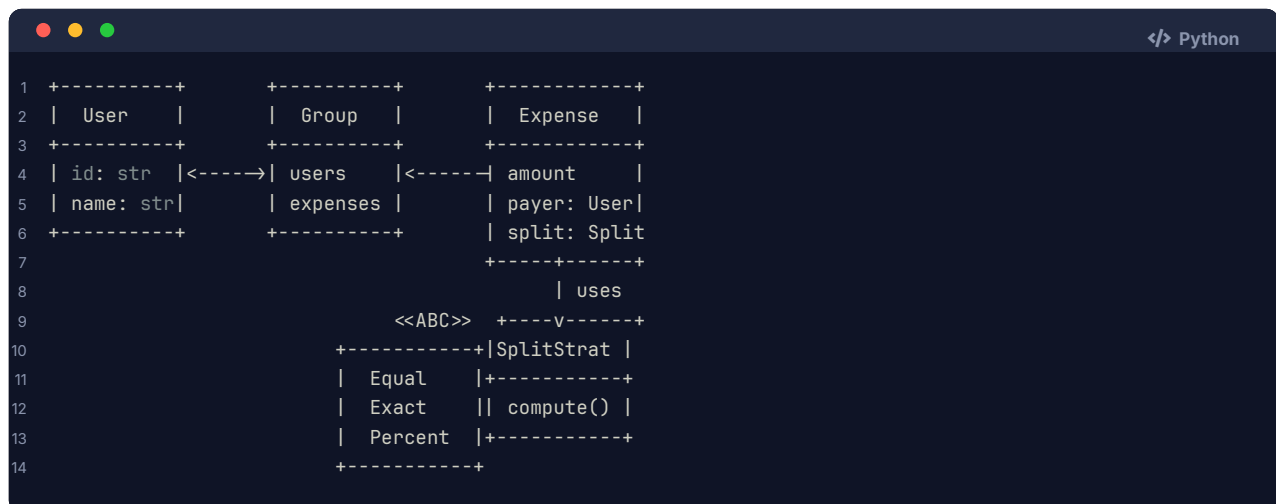
Clarifying questions to ask

- › Currency? (Single currency; mention multi-currency extension.)
- › Can the payer be a participant? (Yes — they effectively owe zero or receive money.)
- › Simplify debts: across whole app or per-group? (Per-group for now.)
- › Partial payments / settle-up? (Yes — add expense with type = SETTLE.)
- › Persistence? (In-memory; mention DB extension.)

5.2 Core entities / classes

Class	Responsibility
User	Name, id, personal balance map
Group	Collection of users + expenses
Expense	Amount, payer, participants, split strategy
SplitStrategy (ABC)	<code>compute_shares(amount, participants) → dict[User, float]</code>
EqualSplit	Divide evenly
ExactSplit	Caller specifies each person's share
PercentSplit	Caller specifies each person's percentage
Balance	Utility: compute net balances + simplify

5.3 Class diagram (ASCII UML)



5.4 Design patterns used

- › **Strategy** — SplitStrategy for different split computations.
- › **Domain Model** — User, Group, Expense map directly to business entities.

5.5 Full code

```

1  from __future__ import annotations
2
3  from abc import ABC, abstractmethod
4  from dataclasses import dataclass, field
5  from enum import Enum, auto
6  from typing import Optional
7  import uuid
8
9
10 # ----- #
11 # Split Strategies                                     #
12 # ----- #
13
14 class SplitStrategy(ABC):
15     @abstractmethod
16     def compute_shares(
17         self, amount: float, participants: list[User], **kwargs
18     ) → dict[str, float]:
19         """Return {user_id: share_amount} for each participant."""
20         ...
21
22
23 class EqualSplit(SplitStrategy):
24     def compute_shares(
25         self, amount: float, participants: list[User], **kwargs
26     ) → dict[str, float]:
27         share = round(amount / len(participants), 2)
28         result = {u.user_id: share for u in participants}
29         # Correct rounding error on first participant
30         total = sum(result.values())
31         diff = round(amount - total, 2)
32         first = participants[0].user_id
33         result[first] = round(result[first] + diff, 2)
  
```

```

34         return result
35
36
37 class ExactSplit(SplitStrategy):
38     def compute_shares(
39         self, amount: float, participants: list[User], **kwargs
40     ) → dict[str, float]:
41         shares: dict[str, float] = kwargs.get("exact_shares", {})
42         if abs(sum(shares.values()) - amount) > 0.01:
43             raise ValueError("Exact shares must sum to total amount")
44         return {u.user_id: shares[u.user_id] for u in participants}
45
46
47 class PercentSplit(SplitStrategy):
48     def compute_shares(
49         self, amount: float, participants: list[User], **kwargs
50     ) → dict[str, float]:
51         percents: dict[str, float] = kwargs.get("percents", {})
52         if abs(sum(percents.values()) - 100) > 0.01:
53             raise ValueError("Percentages must sum to 100")
54         return {
55             u.user_id: round(amount * percents[u.user_id] / 100, 2)
56             for u in participants
57         }
58
59
60 # ----- #
61 # User                                         #
62 # ----- #
63
64 @dataclass
65 class User:
66     name: str
67     user_id: str = field(default_factory=lambda: str(uuid.uuid4())[:8])
68
69     def __hash__(self) → int:
70         return hash(self.user_id)
71
72     def __eq__(self, other: object) → bool:
73         return isinstance(other, User) and self.user_id == other.user_id
74
75     def __repr__(self) → str:
76         return f"User({self.name})"
77
78
79 # ----- #
80 # Expense                                     #
81 # ----- #
82
83 class SplitType(Enum):
84     EQUAL = auto()
85     EXACT = auto()
86     PERCENT = auto()
87
88
89 @dataclass
90 class Expense:
91     expense_id: str
92     description: str
93     amount: float
94     payer: User

```

```

95     participants: list[User]
96     split_strategy: SplitStrategy
97     shares: dict[str, float] # user_id → amount owed by that user
98
99
100 # ----- #
101 # Group                                     #
102 # ----- #
103
104 class Group:
105     def __init__(self, name: str) → None:
106         self.group_id = str(uuid.uuid4())[:8]
107         self.name = name
108         self.members: list[User] = []
109         self.expenses: list[Expense] = []
110         # net_balances[a][b] = amount b owes a (positive = b owes a)
111         self._net: dict[str, dict[str, float]] = {}
112
113     def add_member(self, user: User) → None:
114         if user not in self.members:
115             self.members.append(user)
116             self._net.setdefault(user.user_id, {})
117
118     def add_expense(
119         self,
120         description: str,
121         amount: float,
122         payer: User,
123         participants: list[User],
124         strategy: SplitStrategy,
125         **kwargs,
126     ) → Expense:
127         if payer not in self.members:
128             raise ValueError(f"{payer} is not a group member")
129         shares = strategy.compute_shares(amount, participants, **kwargs)
130         expense = Expense(
131             expense_id=str(uuid.uuid4())[:8],
132             description=description,
133             amount=amount,
134             payer=payer,
135             participants=participants,
136             split_strategy=strategy,
137             shares=shares,
138         )
139         self.expenses.append(expense)
140         self._update_balances(expense)
141         return expense
142
143     def _update_balances(self, expense: Expense) → None:
144         payer_id = expense.payer.user_id
145         for uid, share in expense.shares.items():
146             if uid == payer_id:
147                 continue # payer doesn't owe themselves
148             # uid owes payer_id `share` amount
149             self._net.setdefault(payer_id, {})
150             self._net.setdefault(uid, {})
151             current = self._net[payer_id].get(uid, 0)
152             self._net[payer_id][uid] = round(current + share, 2)
153
154     def show_balances(self) → None:
155         print(f"\n--- Balances in group '{self.name}' ---")

```

```

156     any_balance = False
157     for creditor_id, debtors in self._net.items():
158         creditor_name = self._user_name(creditor_id)
159         for debtor_id, amount in debtors.items():
160             net = amount - self._net.get(debtor_id, {}).get(creditor_id, 0)
161             if net > 0.01:
162                 any_balance = True
163                 debtor_name = self._user_name(debtor_id)
164                 print(f" {debtor_name} owes {creditor_name}: ${net:.2f}")
165     if not any_balance:
166         print(" All settled up!")
167
168     def simplify_debts(self) → list[tuple[str, str, float]]:
169         """
170         Minimize number of transactions using a greedy creditor-debtor matching.
171         Returns list of (debtor_name, creditor_name, amount).
172         """
173         # Compute net position for each user
174         net_position: dict[str, float] = {m.user_id: 0.0 for m in self.members}
175         for creditor_id, debtors in self._net.items():
176             for debtor_id, amount in debtors.items():
177                 net_position[creditor_id] = round(net_position[creditor_id] + amount, 2)
178                 net_position[debtor_id] = round(net_position[debtor_id] - amount, 2)
179
180         # Separate into creditors (positive) and debtors (negative)
181         creditors = {uid: amt for uid, amt in net_position.items() if amt > 0.01}
182         debtors = {uid: -amt for uid, amt in net_position.items() if amt < -0.01}
183
184         transactions: list[tuple[str, str, float]] = []
185         cred_list = sorted(creditors.items(), key=lambda x: -x[1])
186         debt_list = sorted(debtors.items(), key=lambda x: -x[1])
187
188         i, j = 0, 0
189         while i < len(cred_list) and j < len(debt_list):
190             cid, c_amt = cred_list[i]
191             did, d_amt = debt_list[j]
192             settled = round(min(c_amt, d_amt), 2)
193             transactions.append((self._user_name(did), self._user_name(cid), settled))
194             c_amt = round(c_amt - settled, 2)
195             d_amt = round(d_amt - settled, 2)
196             cred_list[i] = (cid, c_amt)
197             debt_list[j] = (did, d_amt) # dummy; just advance index
198             if c_amt < 0.01:
199                 i += 1
200             if d_amt < 0.01:
201                 j += 1
202
203         return transactions
204
205     def _user_name(self, user_id: str) → str:
206         for m in self.members:
207             if m.user_id == user_id:
208                 return m.name
209         return user_id
210
211     if __name__ == "__main__":
212         alice = User("Alice")
213         bob = User("Bob")
214         charlie = User("Charlie")
215
216

```

```

217     group = Group("Trip to Goa")
218     for u in [alice, bob, charlie]:
219         group.add_member(u)
220
221     # Alice pays Rs 300 split equally
222     group.add_expense(
223         "Dinner", 300.0, alice, [alice, bob, charlie], EqualSplit()
224     )
225
226     # Bob pays Rs 200: Charlie pays 50%, Bob 30%, Alice 20%
227     group.add_expense(
228         "Cab", 200.0, bob,
229         [alice, bob, charlie], PercentSplit(),
230         percents={alice.user_id: 20, bob.user_id: 30, charlie.user_id: 50}
231     )
232
233     group.show_balances()
234
235     print("\n--- Simplified transactions ---")
236     for debtor, creditor, amount in group.simplify_debts():
237         print(f" {debtor} pays {creditor}: ${amount:.2f}")

```

5.6 Extensibility discussion

- › **New split type (shares-based):** Add `SharesSplit(SplitStrategy)` where each participant has N shares; their fraction = $N_i / \text{sum}(N)$. Zero changes to `Group`.
- › **Multi-currency:** Add `currency: str` to `Expense`; convert to a base currency using an injected `CurrencyConverter` before updating balances.
- › **Settle-up:** `settle(payer, payee, amount)` is just `add_expense("Settlement", amount, payer, [payer, payee], ExactSplit(), exact_shares={payer.user_id:0, payee.user_id:amount})`.
- › **Persistence:** Replace `self._net` updates with DB writes. The `Group` class becomes a service layer calling a repository.

Interview takeaway: The debt-simplification algorithm is a classic greedy: compute net positions, then greedily match biggest creditor with biggest debtor. It's $O(N \log N)$ and minimizes the number of transactions (though not necessarily the amounts).

6 5. Elevator System

6.1 Requirements

Functional

- › Multiple elevators in a building.
- › External requests: `request_floor(floor, direction)` — someone on floor 3 wants to go UP.
- › Internal requests: `select_floor(elevator_id, floor)` — someone inside presses a button.
- › Scheduler assigns best elevator to each external request.
- › Each elevator has states: IDLE, MOVING_UP, MOVING_DOWN, MAINTENANCE.

Clarifying questions to ask

- › How many elevators / floors? (Configurable.)
- › Any elevator capacity / weight limit? (Mention, skip implementation.)
- › Priority floors (emergency)? (Out of scope for now.)
- › Which scheduling algorithm? (SCAN / "look" algorithm — service in current direction first.)

- › Real-time simulation or just state transitions? (State transitions.)

6.2 Core entities / classes

Class	Responsibility
ElevatorState (enum)	IDLE, MOVING_UP, MOVING_DOWN, MAINTENANCE
Direction (enum)	UP, DOWN, IDLE
Request	Floor + direction (external) or just floor (internal)
Elevator	Single car; state machine; queue of floors to serve
Scheduler (ABC)	Assigns external request to an elevator
ScanScheduler	SCAN/LOOK algorithm implementation
ElevatorSystem	Owns elevators + scheduler; top-level API

6.3 Class diagram (ASCII UML)

```

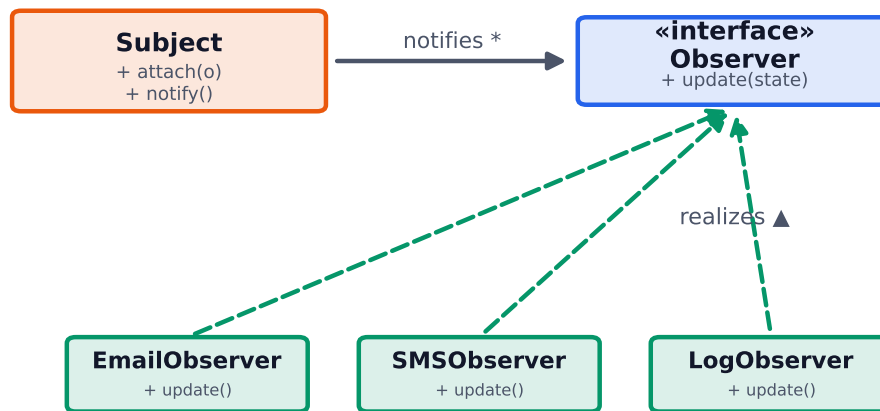
+-----+
| ElevatorSystem |
+-----+
| elevators: list |
| scheduler: Scheduler |
+-----+
| request(floor, dir) |
| select_floor(eid,f) |
| step() |
+-----+
|
+-----+-----+ <<ABC>>
| Elevator | +-----+
+-----+ | Scheduler |
| id: int | +-----+
| floor: int | | assign() |
| state: State | +-----+
| destinations | |
+-----+ +-----V-----+
| ScanSched |
+-----+

```

6.4 Design patterns used

- › **State** — ElevatorState drives behavior. The transition table (IDLE → MOVING_UP, etc.) is explicit.
- › **Strategy** — Scheduler is pluggable (SCAN, Nearest-Car, Zone-based).
- › **Observer** (mention) — In production, elevators would publish state-change events; floors would subscribe to update displays.

Observer pattern — publish/subscribe decoupling



subject broadcasts state changes; observers subscribe/unsubscribe at runtime

Figure 1. Observer lets a subject notify many observers of state changes without knowing their concrete types — the basis of event systems and reactive UIs.

6.5 Full code

```

1  from __future__ import annotations
2
3  from abc import ABC, abstractmethod
4  from enum import Enum, auto
5  from dataclasses import dataclass, field
6  from typing import Optional
7  import heapq
8
9
10 # ----- #
11 # Enums                                     #
12 # ----- #
13
14 class ElevatorState(Enum):
15     IDLE = auto()
16     MOVING_UP = auto()
17     MOVING_DOWN = auto()
18     MAINTENANCE = auto()
19
20
21 class Direction(Enum):
22     UP = auto()
23     DOWN = auto()
24     IDLE = auto()
25
26
27 # ----- #
28 # Request                                   #
29 # ----- #
  
```

```

30
31 @dataclass
32 class Request:
33     floor: int
34     direction: Direction # Direction.IDLE for internal (in-elevator) requests
35     is_internal: bool = False
36
37     def __repr__(self) → str:
38         kind = "INT" if self.is_internal else f"EXT-{self.direction.name}"
39         return f"Req(floor={self.floor},{kind})"
40
41
42 # ----- #
43 # Elevator #
44 # ----- #
45
46 class Elevator:
47     """
48     Single elevator car.
49
50     Internally maintains two min-heaps:
51     - up_queue: floors > current floor, to serve while going up (min-heap)
52     - down_queue: floors < current floor, to serve while going down (max-heap via negation)
53     SCAN algorithm: exhaust current direction first, then reverse.
54     """
55
56     def __init__(self, elevator_id: int, initial_floor: int = 1) → None:
57         self.elevator_id = elevator_id
58         self.current_floor = initial_floor
59         self.state = ElevatorState.IDLE
60         self.direction = Direction.IDLE
61         self._up_queue: list[int] = [] # min-heap
62         self._down_queue: list[int] = [] # max-heap (store negatives)
63
64     def add_destination(self, floor: int) → None:
65         if self.state == ElevatorState.MAINTENANCE:
66             raise RuntimeError(f"Elevator {self.elevator_id} is under maintenance")
67         if floor == self.current_floor:
68             return
69         if floor > self.current_floor:
70             if floor not in self._up_queue:
71                 heapq.heappush(self._up_queue, floor)
72         else:
73             if -floor not in self._down_queue:
74                 heapq.heappush(self._down_queue, -floor)
75         self._update_state()
76
77     def step(self) → Optional[int]:
78         """
79         Move one floor toward the next destination.
80         Returns the floor reached, or None if already at destination / idle.
81         """
82         if self.state == ElevatorState.IDLE or self.state == ElevatorState.MAINTENANCE:
83             return None
84
85         if self.state == ElevatorState.MOVING_UP:
86             if self._up_queue:
87                 next_floor = self._up_queue[0]
88                 self.current_floor += 1
89                 print(f" Elevator {self.elevator_id} ↑ Floor {self.current_floor}")
90                 if self.current_floor == next_floor:

```

```

91         heapq.heappop(self._up_queue)
92         print(f" Elevator {self.elevator_id} STOP at Floor {self.current_floor}")
93         self._update_state()
94
95     elif self.state == ElevatorState.MOVING_DOWN:
96         if self._down_queue:
97             next_floor = -self._down_queue[0]
98             self.current_floor -= 1
99             print(f" Elevator {self.elevator_id} ↓ Floor {self.current_floor}")
100             if self.current_floor == next_floor:
101                 heapq.heappop(self._down_queue)
102                 print(f" Elevator {self.elevator_id} STOP at Floor {self.current_floor}")
103                 self._update_state()
104
105     return self.current_floor
106
107 def _update_state(self) → None:
108     """Transition state based on queues and current direction."""
109     if not self._up_queue and not self._down_queue:
110         self.state = ElevatorState.IDLE
111         self.direction = Direction.IDLE
112     elif self.direction == Direction.UP:
113         if self._up_queue:
114             self.state = ElevatorState.MOVING_UP
115         else:
116             # Exhausted upward requests; switch down
117             self.direction = Direction.DOWN
118             self.state = ElevatorState.MOVING_DOWN
119     elif self.direction == Direction.DOWN:
120         if self._down_queue:
121             self.state = ElevatorState.MOVING_DOWN
122         else:
123             self.direction = Direction.UP
124             self.state = ElevatorState.MOVING_UP
125     else:
126         # Was IDLE; pick direction based on first pending request
127         if self._up_queue and (
128             not self._down_queue or self._up_queue[0] ≥ self.current_floor
129         ):
130             self.direction = Direction.UP
131             self.state = ElevatorState.MOVING_UP
132         else:
133             self.direction = Direction.DOWN
134             self.state = ElevatorState.MOVING_DOWN
135
136 def cost_to_serve(self, floor: int, direction: Direction) → int:
137     """
138     Heuristic cost for scheduler: number of steps to reach `floor`.
139     Considers current direction and queued stops.
140     """
141     if self.state == ElevatorState.MAINTENANCE:
142         return float("inf") # type: ignore[return-value]
143
144     if self.state == ElevatorState.IDLE:
145         return abs(self.current_floor - floor)
146
147     if self.state == ElevatorState.MOVING_UP:
148         if floor ≥ self.current_floor and direction == Direction.UP:
149             return floor - self.current_floor
150         # Must finish upward, come back
151         top = max(self._up_queue) if self._up_queue else self.current_floor

```

```

152         return (top - self.current_floor) + (top - floor)
153
154     if self.state == ElevatorState.MOVING_DOWN:
155         if floor ≤ self.current_floor and direction == Direction.DOWN:
156             return self.current_floor - floor
157         bottom = min(-x for x in self._down_queue) if self._down_queue else self.current_floor
158         return (self.current_floor - bottom) + (floor - bottom)
159
160     return abs(self.current_floor - floor)
161
162     def __repr__(self) → str:
163         return (
164             f"Elevator[{self.elevator_id}|Floor {self.current_floor}|"
165             f"{self.state.name}|up={list(self._up_queue)}|"
166             f"down=[{-x for x in self._down_queue}]"
167         )
168
169 # ----- #
170 # Scheduler                                     #
171 # ----- #
172
173 class Scheduler(ABC):
174     @abstractmethod
175     def assign(self, elevators: list[Elevator], request: Request) → Elevator:
176         ...
177
178 class NearestCarScheduler(Scheduler):
179     """Assign the elevator with the lowest cost_to_serve heuristic."""
180
181     def assign(self, elevators: list[Elevator], request: Request) → Elevator:
182         available = [e for e in elevators if e.state ≠ ElevatorState.MAINTENANCE]
183         if not available:
184             raise RuntimeError("No elevators available")
185         return min(
186             available,
187             key=lambda e: e.cost_to_serve(request.floor, request.direction)
188         )
189
190 # ----- #
191 # Elevator System                               #
192 # ----- #
193
194 class ElevatorSystem:
195     def __init__(
196         self,
197         num_elevators: int,
198         num_floors: int,
199         scheduler: Optional[Scheduler] = None,
200     ) → None:
201         self.num_floors = num_floors
202         self.elevators = [Elevator(i + 1) for i in range(num_elevators)]
203         self.scheduler = scheduler or NearestCarScheduler()
204
205     def external_request(self, floor: int, direction: Direction) → None:
206         """Someone on `floor` pressed the UP or DOWN button."""
207         self._validate_floor(floor)
208         request = Request(floor=floor, direction=direction)
209         elevator = self.scheduler.assign(self.elevators, request)

```

```

213     elevator.add_destination(floor)
214     print(f"    Assigned {request} to Elevator {elevator.elevator_id}")
215
216     def internal_request(self, elevator_id: int, floor: int) → None:
217         """Passenger inside elevator presses button for `floor`."""
218         self._validate_floor(floor)
219         elevator = self._get_elevator(elevator_id)
220         elevator.add_destination(floor)
221         print(f"    Elevator {elevator_id}: internal request for floor {floor}")
222
223     def step_all(self, steps: int = 1) → None:
224         """Simulate `steps` time ticks."""
225         for _ in range(steps):
226             for elevator in self.elevators:
227                 elevator.step()
228
229     def status(self) → None:
230         print("\n--- Elevator Status ---")
231         for e in self.elevators:
232             print(f"    {e}")
233
234     def _validate_floor(self, floor: int) → None:
235         if not (1 ≤ floor ≤ self.num_floors):
236             raise ValueError(f"Floor {floor} out of range [1, {self.num_floors}]")
237
238     def _get_elevator(self, elevator_id: int) → Elevator:
239         for e in self.elevators:
240             if e.elevator_id == elevator_id:
241                 return e
242             raise ValueError(f"Elevator {elevator_id} not found")
243
244
245 if __name__ == "__main__":
246     system = ElevatorSystem(num_elevators=2, num_floors=10)
247     system.status()
248
249     print("\n--- Requests ---")
250     system.external_request(5, Direction.UP) # person on floor 5 wants to go up
251     system.external_request(3, Direction.DOWN) # person on floor 3 wants to go down
252     system.internal_request(1, 8) # inside elevator 1, press 8
253
254     system.status()
255
256     print("\n--- Simulating 10 steps ---")
257     system.step_all(steps=10)
258     system.status()

```

6.6 Extensibility discussion

- **New scheduler (zone-based):** Implement `ZoneScheduler(Scheduler)` that restricts each elevator to a floor range. Inject at construction.
- **Emergency mode:** Add `ElevatorState.EMERGENCY`. Override `_update_state` to route all cars to ground floor first.
- **Capacity tracking:** Add `current_load: int` and `max_capacity: int` to `Elevator`. Scheduler skips full elevators.
- **Floor display / observer:** Elevators publish `FloorReachedEvent`; `FloorDisplay` objects subscribe and update arrow indicators.

Interview takeaway: The SCAN algorithm (also called "elevator algorithm") is exactly

how real elevator schedulers work — it's the same algorithm used by disk schedulers. Name it. The state machine with direction + two queues (up/down) is the key implementation insight.

7 6. Tic-Tac-Toe (Clean OO Board Game)

7.1 Requirements

Functional

- › Two players, configurable board size (default 3×3).
- › `make_move(player, row, col)` — place mark; validate; check win/draw.
- › Win condition: any row, column, or diagonal fully owned by one player.
- › Draw: board full, no winner.
- › Support pluggable players (human, AI/random).

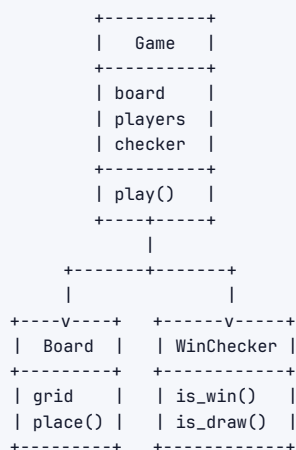
Clarifying questions to ask

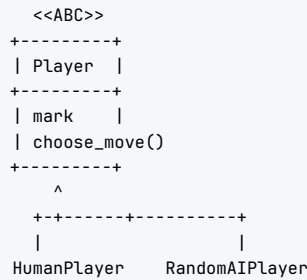
- › Board size always 3×3 or configurable? (Configurable; win requires N-in-a-row.)
- › Two players only, or more? (Two for now; mention extension.)
- › AI player? (Optional; use Strategy.)
- › Can the same player go twice? (No — enforce turn order.)
- › Replay / undo moves? (Out of scope; mention Command pattern for undo.)

7.2 Core entities / classes

Class	Responsibility
Mark (enum)	X, O, EMPTY
Board	Grid state; place/validate mark; check win/draw
Player (ABC)	Name, mark, <code>choose_move(board) → (row, col)</code>
HumanPlayer	Reads from provided input (injectable for testing)
RandomAIPlayer	Picks a random empty cell
WinChecker	Encapsulates win-detection logic (single responsibility)
Game	Orchestrates turn loop; holds players + board

7.3 Class diagram (ASCII UML)





7.4 Design patterns used

- › **Strategy** — `Player.choose_move` makes each player type a strategy for move selection.
- › **Template Method** — `Game.play()` defines the fixed turn loop; player-specific behavior is deferred to `choose_move`.
- › **Single Responsibility** — `WinChecker` is separate from `Board` (board manages state; checker evaluates it).
- › **Command** (mention) — For undo, wrap each move in a `Command` object with `execute/undo`.

7.5 Full code

```

1  from __future__ import annotations
2
3  import random
4  from abc import ABC, abstractmethod
5  from enum import Enum, auto
6  from typing import Optional
7
8
9  # ----- #
10 # Mark                                     #
11 # ----- #
12
13 class Mark(Enum):
14     EMPTY = "."
15     X = "X"
16     O = "O"
17
18
19 # ----- #
20 # Board                                     #
21 # ----- #
22
23 class Board:
24     def __init__(self, size: int = 3) → None:
25         self.size = size
26         self.grid: list[list[Mark]] = [
27             [Mark.EMPTY] * size for _ in range(size)
28         ]
29         self._moves_made = 0
30
31     def place(self, row: int, col: int, mark: Mark) → None:
32         if not (0 ≤ row < self.size and 0 ≤ col < self.size):
33             raise ValueError(f"Position ({row},{col}) out of bounds")
34         if self.grid[row][col] ≠ Mark.EMPTY:
35             raise ValueError(f"Position ({row},{col}) already occupied")
36         self.grid[row][col] = mark
  
```

```

37         self._moves_made += 1
38
39     def is_full(self) → bool:
40         return self._moves_made == self.size * self.size
41
42     def empty_cells(self) → list[tuple[int, int]]:
43         return [
44             (r, c)
45             for r in range(self.size)
46             for c in range(self.size)
47             if self.grid[r][c] == Mark.EMPTY
48         ]
49
50     def display(self) → None:
51         header = " " + " ".join(str(c) for c in range(self.size))
52         print(header)
53         for r, row in enumerate(self.grid):
54             print(f"{r} " + " ".join(cell.value for cell in row))
55         print()
56
57
58 # ----- #
59 # Win Checker #
60 # ----- #
61
62 class WinChecker:
63     """Stateless win-detection. Separated from Board (SRP)."""
64
65     def is_winner(self, board: Board, mark: Mark) → bool:
66         size = board.size
67         g = board.grid
68
69         # Rows
70         for row in g:
71             if all(cell == mark for cell in row):
72                 return True
73
74         # Columns
75         for col in range(size):
76             if all(g[row][col] == mark for row in range(size)):
77                 return True
78
79         # Main diagonal
80         if all(g[i][i] == mark for i in range(size)):
81             return True
82
83         # Anti-diagonal
84         if all(g[i][size - 1 - i] == mark for i in range(size)):
85             return True
86         return False
87
88     def is_draw(self, board: Board) → bool:
89         return board.is_full()
90
91 # ----- #
92 # Players #
93 # ----- #
94
95 class Player(ABC):
96     def __init__(self, name: str, mark: Mark) → None:
97         self.name = name
98         self.mark = mark

```

```

98     @abstractmethod
99     def choose_move(self, board: Board) → tuple[int, int]:
100         ...
101
102     def __repr__(self) → str:
103         return f"Player({self.name},{self.mark.value})"
104
105
106 class HumanPlayer(Player):
107     """In tests, inject a moves iterator so no stdin needed."""
108
109     def __init__(
110         self,
111         name: str,
112         mark: Mark,
113         moves_iter: Optional[iter] = None, # type: ignore[type-arg]
114     ) → None:
115         super().__init__(name, mark)
116         self._moves_iter = moves_iter
117
118     def choose_move(self, board: Board) → tuple[int, int]:
119         if self._moves_iter is not None:
120             return next(self._moves_iter)
121         while True:
122             try:
123                 row = input(f"{self.name} ({self.mark.value}), enter row col: ")
124                 row, col = map(int, row.strip().split())
125                 return row, col
126             except (ValueError, EOFError):
127                 print(" Invalid input. Enter two integers: row col")
128
129
130 class RandomAIPlayer(Player):
131     def __init__(self, name: str, mark: Mark, seed: Optional[int] = None) → None:
132         super().__init__(name, mark)
133         self._rng = random.Random(seed)
134
135     def choose_move(self, board: Board) → tuple[int, int]:
136         empties = board.empty_cells()
137         if not empties:
138             raise RuntimeError("No moves available")
139         choice = self._rng.choice(empties)
140         print(f" {self.name} plays ({choice[0]},{choice[1]})")
141         return choice
142
143
144 # ----- #
145 # Game                                         #
146 # ----- #
147
148 class GameResult(Enum):
149     WIN = auto()
150     DRAW = auto()
151     IN_PROGRESS = auto()
152
153
154 class Game:
155     """
156     Orchestrates the turn loop.
157     Decoupled from I/O via injectable players.
158     """

```

```

159
160     def __init__(
161         self,
162         player1: Player,
163         player2: Player,
164         board_size: int = 3,
165     ) → None:
166         if player1.mark == player2.mark:
167             raise ValueError("Players must have different marks")
168         self.players = [player1, player2]
169         self.board = Board(size=board_size)
170         self.checker = WinChecker()
171         self._current_idx = 0
172         self.winner: Optional[Player] = None
173
174     @property
175     def current_player(self) → Player:
176         return self.players[self._current_idx]
177
178     def play_turn(self) → GameResult:
179         player = self.current_player
180         row, col = player.choose_move(self.board)
181         self.board.place(row, col, player.mark)
182         self.board.display()
183
184         if self.checker.is_winner(self.board, player.mark):
185             self.winner = player
186             print(f" {player.name} wins!")
187             return GameResult.WIN
188
189         if self.checker.is_draw(self.board):
190             print(" It's a draw!")
191             return GameResult.DRAW
192
193         self._current_idx = 1 - self._current_idx
194         return GameResult.IN_PROGRESS
195
196     def play(self) → Optional[Player]:
197         """Run the full game loop. Returns winner or None on draw."""
198         self.board.display()
199         while True:
200             result = self.play_turn()
201             if result != GameResult.IN_PROGRESS:
202                 break
203         return self.winner
204
205
206 if __name__ == "__main__":
207     # Scripted demo: two AI players with fixed seeds for reproducibility
208     p1 = RandomAIPlayer("Alice", Mark.X, seed=42)
209     p2 = RandomAIPlayer("Bob", Mark.O, seed=99)
210
211     game = Game(p1, p2, board_size=3)
212     winner = game.play()
213     if winner:
214         print(f"Winner: {winner.name}")
215     else:
216         print("Game ended in a draw")
217
218     # Human vs AI example (uncomment for interactive play):
219     # human = HumanPlayer("You", Mark.X)

```

```

220     # ai = RandomAIPlayer("Bot", Mark.0)
221     # Game(human, ai).play()

```

7.6 Extensibility discussion

- **N-in-a-row win (Connect-4 style):** Parameterize WinChecker with win_length < size. Only the checker changes.
- **More than 2 players:** Change self.players to a list; cycle with (current_idx + 1) % len(players). Board already supports arbitrary marks.
- **AI with Minimax:** Replace RandomAIPlayer.choose_move with a minimax search tree. The Player interface is unchanged.
- **Undo last move:** Wrap board.place in a MoveCommand(Command). game.undo() calls command.undo() which clears the cell and decrements _moves_made.
- **Networked game:** RemotePlayer(Player) overrides choose_move to send the board state over a socket and wait for a response.

Interview takeaway: The WinChecker separation is a key OOP point — "Board manages state; WinChecker evaluates it." This satisfies SRP and makes unit testing trivial. Always mention the injectable moves iterator for testing HumanPlayer without stdin.

8 Common Patterns Across LLD Problems

8.1 Entity-extraction checklist

When you get a new problem, run through this in 5 minutes:

1. **Nouns → candidate classes:** Underline every noun. Parking Lot → ParkingLot, Level, Spot, Vehicle, Ticket, Fee.
2. **Verbs → candidate methods:** Circle every verb. "find available spot" → find_spot(vehicle).
3. **Prune attributes:** Nouns that are just data fields (name, price, timestamp) become fields, not classes.
4. **Find the variability:** What changes? Pricing? Split type? Scheduling algorithm? That variability → Strategy pattern.
5. **Find the lifecycle:** What has state? Elevator (Idle → Moving), Ticket (Open → Closed) → State pattern or enum.
6. **Find the singleton:** What is there exactly one of? ParkingLot, ElevatorSystem → Singleton.
7. **Find the family:** Vehicles, Players, Strategies → Factory + ABC.

8.2 Which pattern fits which need

Need	Pattern	Example in this file
Swap algorithm at runtime	Strategy	Pricing, Split, Scheduler, Rate Limiter, Player
Object with lifecycle phases	State	Elevator (IDLE/MOVING/MAINTENANCE)
One global instance	Singleton	ParkingLot, ElevatorSystem
Create family of objects	Factory Method	VehicleFactory
Fixed algorithm, variable steps	Template Method	Vehicle.allowed_spot_types, Game.play
Undo / replayable actions	Command	Board moves (TicTacToe extension)

Need	Pattern	Example in this file
Notify many on change	Observer	ElevatorSystem → FloorDisplay
Wrap object with extra behavior	Decorator	Rate limiter middleware layer

8.3 Revision cheat sheet

LRU Cache	→ HashMap + DLinkedList + sentinel head/tail
Parking Lot	→ Strategy(pricing) + Factory(vehicle) + Singleton(lot)
Rate Limiter	→ Strategy(algorithm) + per-client state
Splitwise	→ Strategy(split) + greedy debt simplification
Elevator	→ State machine + Strategy(scheduler) + SCAN algorithm
Tic-Tac-Toe	→ Strategy(player) + SRP (Board vs WinChecker)

8.4 The five questions interviewers ask at the end

1. "How would you make this thread-safe?" → Locks per shared resource; mention `threading.Lock` / `concurrent.futures`.
2. "How would you scale this to 1M users?" → Move to Redis/DB-backed storage; horizontal sharding; mention CAP theorem for rate limiter.
3. "Add feature X — what changes?" → Should be "add a new class, zero changes to existing" (OCP). If it isn't, your design has a seam problem.
4. "How would you test this?" → Unit test each strategy independently; mock dependencies; inject test doubles (e.g., moves iterator in TicTacToe).
5. "What did you deprioritize?" → Show you know the tradeoffs: "I skipped persistence, thread-safety on the outer loop, and error telemetry. For production I'd add X."

Final interview takeaway: In a 45-min round, a clean, running 200-line solution with 4 well-named classes beats a 500-line "enterprise" solution with 12 interfaces that doesn't run. Write the smallest correct design, name the patterns you used, and show one extensibility path. That's the formula.